

Functions In C



C-Functions

- **Functions are blocks of code that perform specific tasks.**
- The programming statements of a function are enclosed within `{ }` braces.
- Provide reusability and modularity.
- Good memory utilization.
- Easy to follow, develop and maintain

Function Aspects

Function
Declaration

Function Definition

Function Call

- **Function Declaration**

Syntax: `return_type function_name(parameter_list);`

- **Function Definition**

Syntax:
`return_type function_name(parameter_list)`
`{`
 `// Function body`
 `return value; // Optional`
`}`

- **Function Call**

Syntax: `function_name(arguments);`

Function Aspects

Function Declaration

Function Declaration (Prototype)

This informs the compiler about the function's name, its return type, and its expected arguments before the function is called.

It acts as a safety contract to ensure arguments are formatted properly.

```
return_type function_name(parameter_type1, parameter_type2);
```

It is also called a **function prototype**.

Example: `int add(int, int);`

This tells the compiler that there is a function named `add` which takes two integers and returns an integer.

Function Aspects

Function Definition

Function Definition

This contains the actual block of logic (body) wrapped inside curly braces {} that executes when the function is invoked.

```
return_type function_name(type param1, type param2)
{
    // Function body / statements
    return value; // Optional depending on return_type
}
```

Example:

```
int add(int a, int b)
{
    int sum;
    sum = a + b;
    return sum;
}
```

Here, the function calculates the sum of two numbers and returns the result.

Function Aspects

Function Call

Once a function is defined, you can use it by simply calling its name followed by parentheses. This tells the program to execute the code inside that function.

A function call means executing the function in a program.

Example:

```
int main() {  
  
    // Function call  
    int result = add(5, 3);  
    printf("The sum is: %d", result);  
    return 0;  
}
```

Call a Function

Declared functions are not executed immediately.

They are "saved for later use", and will be executed when they are called.

To call a function write the function's name followed by **two parentheses ()** and a **semicolon ;**

// Create a function

```
void myFunction() {  
    printf("I just got executed!");  
}
```

```
int main() {  
    myFunction(); // call the function  
    return 0;  
}
```

Example

```
#include <stdio.h>
```

```
void sum();
```

1. Function Declaration
(Prototype)

```
int main()
```

```
{
```

```
    sum();
```

```
    return 0;
```

```
}
```

Function Call

```
void sum()
```

```
{
```

```
    printf("addition operation");
```

```
}
```

Function Definition

Call by value & Call by Reference

Call by value means passing a **copy of the variable's value** to a function, while **call by reference** means passing the **memory address of the variable** using pointers.

Call by Value

When you use call by value, the function creates a ***new block in the stack memory for the formal parameters***. Any arithmetic, updates, or modifications happen entirely inside this isolated memory block.

Call by Reference

Because C lacks native reference variables (unlike C++), you must **pass the memory address using the address-of operator (&)**. The receiving function handles this address with a **pointer (*)** and modifies the value via dereferencing.

Call by value

Actual Parameters: The parameters passed to a function.

Formal Parameters: The parameters received by a function.



Call by value

```
void sum(int ,int );
```

```
int main() {
```

```
    int x=3;
```

```
    int y=4;
```

```
    sum(x,y);
```

```
    printf("inside main values  
are : x is %d y is %d",x,y);
```

```
    return 0;}
```

```
void sum(int x,int y){
```

```
    x=5;
```

```
    y=7;
```

```
    printf("Inside function  
values are : x is %d y is  
%d\n",x,y);
```

```
}
```

Call by Reference

```
int main() {  
    int a = 10, b = 20;  
  
    // Pass by reference  
    swapx(&a, &b);  
  
    printf("Inside main: a =  
%d b = %d", a, b);  
    return 0;  
}
```

```
// Function to swap two  
variables  
// by references  
void swapx(int* x, int* y) {  
    int t;  
    t = *x;  
    *x = *y;  
    *y = t;  
    printf("Inside swapx: x =  
%d y = %d\n", *x, *y);  
}
```

Call by value & Call by Reference

Key Differences at a Glance

Feature	Call by Value	Call by Reference (via Pointers)
Arguments Passed	A duplicate copy of the actual variable data.	The memory address of the actual variable.
Original Modification	Local modifications do not affect the original variable.	Local modifications directly alter the original variable.
Memory Allocation	Uses separate memory locations for actual and formal parameters.	Uses the same memory location via dereferenced addresses.
Use Case	Ideal for small, simple data types where data safety is preferred.	Ideal for large data sets (structures) or when mutating data.

Types of Function In C

Library Function

- Built-in functions (e.g., printf(), scanf(), sqrt()).
- Require specific header files (e.g., stdio.h, math.h).

User Defined Function

- Created by programmers for specific tasks.
- Enhance code reusability and reduce redundancy.

Return Value

Void Function: Does not return a value.

Non-Void Function: Returns a value (e.g., int, float, char).

Example:

```
int getNumber() {  
    return 42;  
}
```

Function Parameters

Information can be passed to functions as a parameter. Parameters act as variables inside the function.

Parameters are specified after the function name, inside the parentheses.

Syntax

```
returnType    functionName(parameter1,    parameter2,  
parameter3)  
  
{  
  
    // code to be executed  
  
}
```

Multiple parameters

Key Points:

- Parameters must be separated by a comma.
- Each parameter needs its own declared data type.
- Arguments passed during function call must match the defined order.

```
void myFunction(char
name[], int age) {
    printf("Hello %s. You are
%d years old.\n", name,
age);
}

int main() {
    myFunction("Liam", 3);
    myFunction("Jenny", 14);
    myFunction("Anja", 30);
    return 0;
}
```

```
void calculateSum(int x, int
y) {
    int sum = x + y;
    printf("The sum of %d + %d
is: %d\n", x, y, sum);
}

int main() {
    calculateSum(5, 3);
    calculateSum(8, 2);
    calculateSum(15, 15);
    return 0;
}
```

Types of User Defined Functions

User defined functions can be divided into four types based on two things:

- Whether the function takes input (arguments)
- Whether the function returns a value

The four types are:

1. No argument, no return value
2. Argument, no return value
3. Argument with return value
4. No argument with return value

1. No Argument, No Return Value

1. No Argument, No Return Value

This type of function **does not take any input** and **does not return any value**.

It simply performs a task and shows the result.

Example:

```
void greet()
{
    printf("Hello, Welcome to C Programming!");
}
int main(){
    greet();
    return 0;
}
```

Explanation: The function greet () does not take any input. It only prints a message. It does not return anything.

2. Argument, No Return Value

2. Argument, No Return Value

This type of function **takes input values**, but **does not return any value**.

It uses the input values to perform a task and directly displays the result.

Example:

```
void sum(int a, int b)
{
    printf("Sum = %d", a + b);
}
int main(){
    sum(5, 7);
    return 0;
}
```

Explanation: The function takes two numbers a and b. It calculates their sum. The result is printed inside the function.

3. Argument with Return Value

3. Argument with Return Value

This type of function takes input values and returns a result.

Example:

```
#include <stdio.h>
int add(int a, int b){
    return a + b;
}int main(){
    int result;
    result = add(4, 6);
    printf("Sum = %d", result);
    return 0;
}
```

Explanation: The function receives two numbers. It calculates their sum. It returns the result to the main function.

4. No Argument with Return Value

4. No Argument with Return Value

This type of function does not take input, but returns a value.

Example:

```
int getNumber(){  
    int num = 10;  
    return num;  
}  
int main(){  
    int n;  
    n = getNumber();  
    printf("Number = %d", n);  
    return 0;  
}
```

Explanation: The function does not take input. It returns a number. The main function receives and prints the value.

Task

Write a program that finds the **square of a number using a function.**

Solution

Write a program that finds the **square of a number using a function**.

```
#include <stdio.h>
int square(int n)
{
    return n * n;
}
int main()
{
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    printf("Square = %d", square(num));
    return 0;
}
```

Task

Write a **Function** to calculate sum of two numbers

Solution

```
#include <stdio.h>

int sum(int a, int b) {

    return a + b;

}

int main() {

    int x, y;

    printf("Enter two numbers: ");

    scanf("%d %d", &x, &y);

    printf("Sum = %d\n", sum(x, y));

    return 0;

}
```

Task

Function to check if number is even or odd

Solution

```
#include <stdio.h>
void isEven(int n) {
    if (n % 2 == 0)
        printf("%d is Even\n", n);
    else
        printf("%d is Odd\n", n);
}
int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    isEven(num);
    return 0;
```

C Variable Scope

In C programming, the scope of a variable refers to the specific region of the program's code where that variable can be directly accessed, referenced, or modified.

The visibility of variables is entirely determined at compile time by their position in the source code layout.

Core Types of Variable Scope

1. Local (Block) Scope
2. Global (File) Scope

Local (Block) Scope

Definition: Any variable declared inside a block of code enclosed by curly braces {}. This includes function bodies, loops (for, while), and conditional blocks (if, switch).

Lifetime: Created when the specific block is entered and destroyed immediately when control exits that block.

Initialization: Not automatically initialized; they contain random garbage data unless explicitly initialized.

Linkage: No linkage; completely invisible to external files and other functions.

Global (File) Scope

Definition: Variables declared outside of all functions, typically at the top of a file.

Lifetime: Persists for the entire duration of the program execution.

Initialization: Automatically initialized to default values by the compiler if not specified (e.g., 0 for integers, 0.0 for floats, and NULL for pointers).

Linkage: External linkage by default. They can be shared across multiple source files using the extern keyword.

Example of local scope

```
void myFunction() {  
    // Local variable that belongs to myFunction  
    int x = 5;  
}  
  
int main() {  
    myFunction();  
    // Print the variable x in the main function  
    printf("%d", x);  
    return 0;  
}
```

Error: 'x' undeclared

Example of global scope

```
// Global variable x
int x = 5;
void myFunction() {
    // We can use x here
    printf("%d", x);
}

int main() {
    myFunction();
    // We can also use x here
    printf("%d", x);
    return 0;
}
```

C Variable Scope

Variable Scoping Comparison

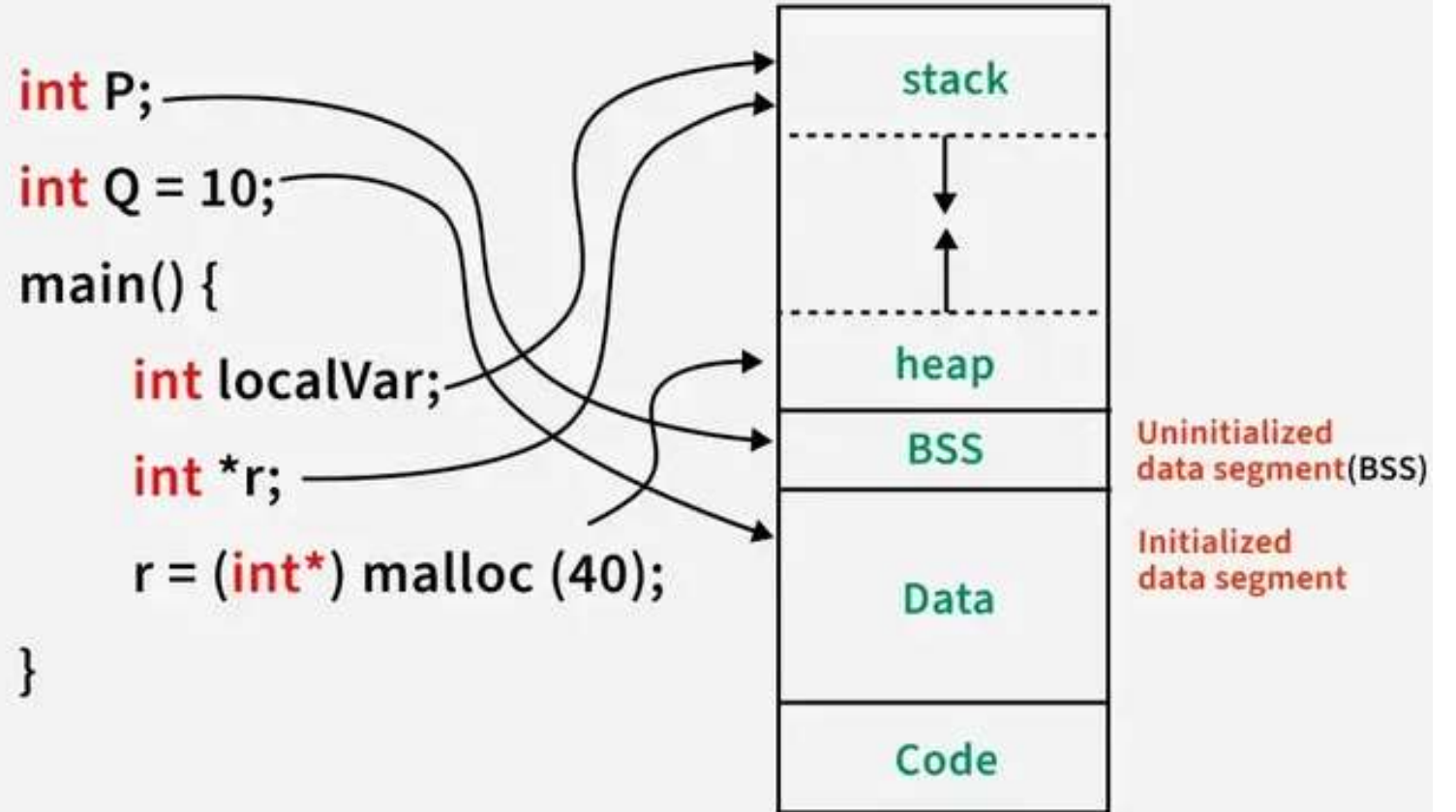
Feature	Local Scope	Global Scope
Declaration Location	Inside <code>{ }</code> braces or function arguments	Outside all functions
Visibility Range	Enclosing block and nested sub-blocks	Everywhere in the file after declaration
Default Initial Value	Garbage value (undefined)	Zero (<code>0</code> , <code>0.0</code> , <code>\0</code> , <code>NULL</code>)
Memory Allocation	Stack memory	Data Segment / BSS Segment
Default Linkage	None	External (accessible in other files)

C Variable Scope

Variable Scoping Comparison

Feature	Local Scope	Global Scope
Declaration Location	Inside { } braces or function arguments	Outside all functions
Visibility Range	Enclosing block and nested sub-blocks	Everywhere in the file after declaration
Default Initial Value	Garbage value (undefined)	Zero (0 , 0.0 , \0 , NULL)
Memory Allocation	Stack memory	Data Segment / BSS Segment
Default Linkage	None	External (accessible in other files)

Memory Layout of C Program



C Math Functions

In C programming, standard mathematical operations require you to include the **<math.h>** header file at the top of your program.

Most of these built-in library functions accept double values as arguments and return a double result.

Essential C Math Functions

Here is a breakdown of the most commonly used math functions in C:

C Math Functions

Essential C Math Functions

Here is a breakdown of the most commonly used math functions in C:

Function	Purpose	Example Syntax	Output
<code>sqrt(x)</code>	Calculates the square root of x	<code>sqrt(16.0)</code>	4.0
<code>cbrt(x)</code>	Calculates the cube root of x	<code>cbrt(27.0)</code>	3.0
<code>pow(x, y)</code>	Raises base x to the power of exponent y (x^y)	<code>pow(4.0, 3.0)</code>	64.0
<code>abs(x)</code>	Absolute value of an integer (Requires <code><stdlib.h></code>)	<code>abs(-10)</code>	10
<code>fabs(x)</code>	Absolute value of a floating-point number	<code>fabs(-4.5)</code>	4.5
<code>ceil(x)</code>	Rounds x upward to the nearest integer	<code>ceil(1.4)</code>	2.0
<code>floor(x)</code>	Rounds x downward to the nearest integer	<code>floor(1.6)</code>	1.0
<code>round(x)</code>	Rounds x to the nearest integer	<code>round(4.6)</code>	5.0
<code>fmod(x, y)</code>	Finds the remainder of x/y (floating-point modulo)	<code>fmod(10.5, 3.0)</code>	1.5

Examples

1. Square Root:

To find the square root of a number, use the `sqrt()` function:

Example

```
printf("%f", sqrt(16));
```

2. Round a Number

The **`ceil()`** function **rounds a number upwards** to its nearest integer, and the **`floor()`** method **rounds a number downwards** to its nearest integer, and returns the result:

Example

```
printf("%f", ceil(1.4));  
printf("%f", floor(1.4));
```

3. Power

The `pow()` function returns the value of x to the power of y (x^y)

Example

```
printf("%f", pow(4, 3));
```

Advantages of Functions

- Avoid rewriting the same logic multiple times.
- Make programs easier to understand and debug.
- Allow for reusability and modularity.
- Divide large programs into manageable parts.
- Functions can be called multiple times from anywhere in the program.



MCQ

What is the purpose of a return statement in a C function?

- a) To exit the program
- b) To send a value back to the function caller
- c) To print a value
- d) To terminate a loop

b) To send a value back to the function caller



MCQ

Which of the following is a correct way to define a function in C?

- a) `int sum(int a, int b) { return a + b; }`
- b) `function sum(int a, int b) { return a + b; }`
- c) `sum(int a, int b): int { return a + b; }`
- d) `int sum(a, b) { return a + b; }`

```
a) int sum(int a, int b)
{ return a + b; }
```



MCQ

What is function prototyping in C?

- a) Writing the body of a function
- b) Creating a reference to a function
- c) Declaring a function before its definition
- d) Calling a function before it is declared

c) Declaring a function before its definition

MCQ

What is the output of the following C code?

```
int add(int a, int b) {  
    return a + b; }  
int main() {  
    printf("%d", add(3, 4));  
    return 0;  
}
```

- a) 3
- b) 4
- c) 7
- d) Error

c) 7



MCQ

What are global variables in C?

- a) Variables declared within a function
- b) Variables that can be accessed by any function in the program
- c) Constants
- d) Variables that store functions

b) Variables that can be accessed by any function in the program



MCQ

What is the output of the following C code?

```
void update(int x) { x = x + 5; }  
int main() {  
    int y = 5;  
    update(y);  
    printf("%d", y);  
    return 0;  
}
```

- a) 5
- b) 10
- c) 0
- d) Error

a) 5

!



MCQ

Which of the following operators is used to access the value pointed to by a pointer?

- a) &
- b) *
- c) ->
- d) ::

b) *

!



MCQ

What does the '&' operator do in C?

- a) It adds two values
- b) It performs a logical AND operation
- c) It gives the address of a variable
- d) It is used to declare a pointer

c) It gives the address of a variable

Recursion

Recursion is the technique of making a function call itself.

Easier to solve complicated problem.

Recursion may be a bit difficult to understand.

The best way to figure out how it works is to experiment with it.

Example

Example

```
int sum(int k);  
int main() {  
    int result = sum(10);  
    printf("%d", result);  
    return 0;  
}  
int sum(int k) {  
    if (k > 0) {  
        return k + sum(k - 1);  
    } else {  
        return 0;  
    }  
}
```

```
10 + sum(9)  
10 + ( 9 + sum(8) )  
10 + ( 9 + ( 8 + sum(7) ) )  
...  
10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + sum(0)  
10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + 0
```

Question

Function to find factorial (using recursion)

Solution

```
#include <stdio.h>
long factorial(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}
int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    printf("Factorial of %d = %ld\n", num, factorial(num));
    return 0;
}
```

Question

WAP to find MAX number in array using function

Solution

```
#include <stdio.h>

int findMax(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max)
            max = arr[i];
    }
    return max;
}

int main() {
    int n;
    printf("Enter size of array: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    printf("Largest element = %d\n", findMax(arr, n));
    return 0;
}
```

Questions

1. WAP to find the Fibonacci series
2. WAP to reverse a number using recursion

Problem Statement

Problem Statement

Create a C program that acts as a menu-driven calculator. The program should offer a user a list of options to perform various mathematical and geometric calculations.

The program should meet the following requirements:

1. **Menu Display:** Present a clear, numbered menu of options to the user.
2. **User Input:** Prompt the user to select an option from the menu.
3. **Calculation Functions:** Use separate functions for each calculation to ensure modularity. The functions should handle the following operations:
 - Calculate the area of a **circle** given its radius.
 - Calculate the area of a **rectangle** given its length and width.
 - Calculate the area of a **square** given its side length.
 - Calculate the **power** (xy) of a number.
 - Calculate the **square root** of a number.
4. **Looping:** The program should continuously display the menu and perform calculations until the user explicitly chooses to exit.
5. **Exit Condition:** An option must be available for the user to gracefully terminate the program.
6. **Error Handling:** The program should handle invalid user input (e.g., a choice not on the menu) and display an appropriate error message. It should also handle specific calculation errors, such as attempting to find the square root of a negative number.

The solution must use **functions** for each calculation, utilize a looping structure (such as **while** or **do-while**) for the menu, and employ a **switch-case** statement to execute the chosen operation.

Problem Statement

```
#include <stdio.h>

#include <math.h> // Required for pow() and sqrt() functions

// Function prototypes
void display_menu();
float area_circle();
float area_rectangle();
int area_square();
double calculate_power();
double calculate_square_root();

int main() {
    int choice;

    // Display the menu and get the first choice *before* the loop
    display_menu();
    printf("Enter your choice or exit: ");
    scanf("%d", &choice);
    printf("\n");

    while (choice != 6) {
        switch (choice) {

            case 1:

                printf("Area of Circle: %.2f\n", area_circle());
```

```
                break;

            case 2:
                printf("Area of Rectangle: %.2f\n", area_rectangle());
                break;

            case 3:
                printf("Area of Square: %d\n", area_square());
                break;

            case 4:
                printf("Result of Power: %.2f\n", calculate_power());
                break;

            case 5:
                printf("Result of Square Root: %.2f\n", calculate_square_root());
                break;

            default:
                printf("Invalid choice. Please try again.\n");
        }
        printf("\n");
    }
```

Problem Statement

```
// Display the menu and get the next choice at the end of each iteration
```

```
display_menu();  
  
printf("Enter your choice: ");  
  
scanf("%d", &choice);  
  
printf("\n");  
  
}
```

```
//The "Exit" message is printed after the loop ends
```

```
printf("Exiting the program. Goodbye!\n");  
  
return 0;}
```

```
// Function to display the menu
```

```
void display_menu() {  
  
    printf("--- Calculator Menu ---\n");  
  
    printf("1. Calculate Area of a Circle\n");  
  
    printf("2. Calculate Area of a Rectangle\n");  
  
    printf("3. Calculate Area of a Square\n");  
  
    printf("4. Calculate Power (x^y)\n");  
  
    printf("5. Calculate Square Root\n");  
  
    printf("6. Exit\n");  
  
    printf("-----\n");}
```

```
// Function to calculate the area of a circle
```

```
float area_circle() {  
  
    float radius;  
  
    printf("Enter the radius of the circle: ");  
  
    scanf("%f", &radius);  
  
    return M_PI * radius * radius;  
  
}
```

```
// Function to calculate the area of a rectangle
```

```
float area_rectangle() {  
  
    float length, width;  
  
    printf("Enter the length and width of the rectangle: ");  
  
    scanf("%f %f", &length, &width);  
  
    return length * width;  
  
}
```

Problem Statement

```
// Function to calculate the area of a square
```

```
int area_square() {  
    int side;  
    printf("Enter the side of the square: ");  
    scanf("%d", &side);  
    return side * side;  
}
```

```
// Function to calculate power using pow()
```

```
double calculate_power() {  
    double base, exponent;  
    printf("Enter the base number: ");  
    scanf("%lf", &base);  
    printf("Enter the exponent: ");  
    scanf("%lf", &exponent);  
    return pow(base, exponent);  
}
```

```
// Function to calculate square root using sqrt()
```

```
double calculate_square_root()  
{  
    double number;  
    printf("Enter a number to find its square root: ");  
    scanf("%lf", &number);  
    if (number < 0) {  
        printf("Error: Cannot calculate the square root of a negative number.\n");  
        return NAN; // Not a Number  
    }  
    return sqrt(number);  
}
```

Problem Statement

Problem Statement

Create a C program for a number guessing game. The program will randomly select an integer between 1 and 100, and the user must guess it. The game should provide hints after each guess until the user finds the correct number.

The program must be structured with the following key components:

1. **Random Number Generation:** Use the `<stdlib.h>` and `<time.h>` libraries. The random number generator must be seeded with `srand(time(0))` to ensure a different number is chosen each time the program runs. The target number should be an integer between 1 and 100, generated using `(rand() % 100) + 1`.
2. **User Input Function:** Implement a function called `getUserGuess()` that is solely responsible for prompting the user for an integer guess and returning it.
3. **Guess Evaluation Function:** Implement a function called `checkGuess()` that accepts the user's guess and the target number as arguments. This function should compare the two numbers and print a hint ("Too high!" or "Too low!"). It should return a flag (e.g., 1 for a correct guess, 0 for an incorrect one) to control the game loop.
4. **Game Loop:** The `main()` function should contain a loop that repeatedly calls the `getUserGuess()` and `checkGuess()` functions until the correct number is guessed.
5. **User Exit Option:** The program should allow the user to exit the game at any time by entering a specific number, such as 0. When the user enters this value, the program should terminate gracefully without proceeding with the guess check.

The program should terminate gracefully once the user has either correctly guessed the number or chosen to exit the game.

Solution

```
#include <stdio.h>
#include <stdlib.h> // For rand() and srand()
#include <time.h> // For time()

// Function to get a guess from the user, returns
// 0 if the user wants to exit
int getUserGuess() {
    int guess;
    printf("Enter your guess (or 0 to exit): ");
    scanf("%d", &guess);
    return guess;
}

// Function to check the guess and provide
// hints
// Returns 1 for a correct guess, 0 for an
// incorrect one
int checkGuess(int guess, int target) {
    if (guess > target) {
        printf("Too high! Try again.\n");
        return 0;
    } else if (guess < target) {
        printf("Too low! Try again.\n");
        return 0;
    } else {
        printf("Congratulations! You guessed the
correct number.\n");
        return 1;
    }
}

int main() {
    srand(time(0)); // Seed the random number
generator
    int target = (rand() % 100) + 1; // Generate a
number between 1 and 100
    int guess;

    printf("I'm thinking of a number between 1 and
100. Can you guess it?\n");

    while (1) { // Loop indefinitely until the user wins or
exits
        guess = getUserGuess();

        // Check if the user wants to exit
        if (guess == 0) {
            printf("Exiting the game. Goodbye!\n");
            break; // Exit the loop
        }

        // Check the guess and break the loop if correct
        if (checkGuess(guess, target)) {
            break; // Exit the loop
        }
    }
    return 0;
}
```